

Similarity-Based Alignment and Generalization

Daniel Oblinger, Vittorio Castelli, Tessa Lau, and Lawrence D. Bergman

IBM T.J. Watson Research, New York
oblio,vittorio,tessalau,bergmanl@us.ibm.com

Abstract. We present a novel approach to learning predictive sequential models, called *similarity-based alignment and generalization*, which incorporates in the induction process a specific form of domain knowledge derived from a similarity function between the points in the input space. When applied to Hidden Markov Models, our framework yields a new class of learning algorithms called *SimAlignGen*. We discuss the application of our approach to the problem of programming by demonstration—the problem of learning a procedural model of user behavior by observing the interaction an application Graphical User Interface (GUI). We describe in detail the SimIOHMM, a specific instance of *SimAlignGen* that extends the known Input-Output Hidden Markov Model (IOHMM). Empirical evaluations of the SimIOHMM show the dependence of the prediction accuracy on the introduced similarity bias, and the computational gains over the IOHMM.

1 Introduction

Many domains require building predictive models from multiple observed data sequences. Examples from the biological domain include protein and DNA sequence alignment or prediction, and from the financial domain include market performance prediction and risk analysis. In the computer networking domain, models of network performance or detection of illegal intrusions have also been learned from observed data sequences. Learning approaches for these domains (like Hidden Markov Model induction [1]) rely primarily on the sequence data itself and utilize little (if any) additional domain knowledge. In this paper we investigate a particular form of domain knowledge that we call similarity knowledge. We show how this knowledge can be employed in learning predictive models from sequential data, and empirically measure the impact of utilizing this additional source of knowledge.

A second thrust of this paper is to present a novel approach for using sequence modeling techniques like HMM learning for the problem of PBD [2, 3] described in Section 2. We will show that, in the PBD domain, similarity knowledge is readily available and that its use improves learning performance

Contributions of this paper include:

- A novel application of traditional sequence alignment algorithms to PBD for learning procedures with complex structure.
- The *SimAlignGen* class of algorithms, an extension of traditional HMMs that

- that adds a similarity function over the input as a new source of bias.
- An instance of an *SimAlignGen* algorithm, the SimIOHMM, implemented as part of a PBD system on the Microsoft Windows platform.
- An empirical evaluation of the SimIOHMM’s ability to learn a real-world procedure from demonstrations and of its significant performance improvement over the IOHMM.

2 An HMM approach to PBD

For the purposes of this paper, we define *programming by demonstration* as the problem of generating a procedure model consistent with a set of demonstrations. Each demonstration is a sequence of events, such as user actions and changes to the application GUIs. A procedure model is consistent with a demonstration if it correctly predicts the actions in the sequence given the prior events in that sequence. Existing PBD systems work well when there is a fixed number of steps in the procedure [4] or when the procedure author can identify the specific step to be generalized [5]. These assumptions are violated when a procedure contains a large number of steps and has complex structure, such as conditional branches. Known sequence learning algorithms like HMMs seem appropriate in these cases since they focus on the problem of identifying optimal sequence alignment.

In this section we briefly outline the primary components necessary for applying these sequence alignment algorithms in the PBD context. A user demonstrates procedures by performing actions, such as clicking the mouse or pressing keyboard keys, on an application’s GUI. In response, the application updates the GUI contents. An *instrumentation component* captures both user and application actions. An *abstraction component* converts the stream of events recorded by the instrumentation into a sequence of *snapshot-action pairs*, called a *trace*. Logically, a snapshot-action pair represents the complete content of the GUI at a point in time, coupled with the action performed by the user. Using the snapshot-action representation we can reduce the problem of learning a procedure to that of predicting the user action from the content of the GUI (and perhaps its history).

The *learning algorithm* combines multiple traces into a procedure model, that can be automatically executed by an *execution component*. The ability of simultaneously combining multiple traces into a procedure model is a feature of our approach that differentiates it from other work in PBD.

3 The *SimAlignGen* family of algorithms

In the PBD domain the visual cues prevalent on GUIs provide a source of knowledge which can be explicitly used to augment the sequence data employed by traditional HMM learning. Typically, an expert can determine how far along a user is in performing a task by observing the content of the visible windows. We provide this visual similarity knowledge to our family of algorithms as a similarity function, which returns a real-valued score measuring

“similarity” between two captured GUI snapshot action pairs (S,A). Formally: **Similarity** : $((S \times A) \times (S \times A)) \rightarrow \mathbb{R}^+$. Thus, instances of the *SimAlignGen* family of algorithms accept an input set of traces, namely, of snapshot-action pairs, and a real-valued similarity function over the snapshot-action pairs. *SimAlignGen* algorithms first align the snapshot-action pairs by simultaneously employing three sources of constraint described below, and then generate an executable procedure model based on the partitioning result. Here, the *alignment* of a set of traces is formally defined as a partition of the snapshot-action pairs. A useful alignment for our purposes is one that groups similar snapshot-action pairs, such that each set of the partition corresponds to what a human would think of as a step in the procedure model. The three sources of constraint are:

1. The alignment should preserve transitions between successive steps. For example, let trace 1 consist of step *A* followed by step *B* and trace 2 consist of step *A'* followed by *B'*; then aligning *A* with *A'* and *B* with *B'* is a good alignment, since it preserves the ordering of transitions within the traces.
2. The alignment should yield sets that can be *generalized*: for each partition set, we should be able to induce a predictive map from snapshot to action.
3. The snapshots in a partition set should be *similar* according to the provided similarity function.

Constructing a learner with the first two biases—transition preservation and generalization—is a difficult problem for which no optimal algorithm exists. One solution consists of iteratively alternating two steps: finding the best alignment of the training data consistent with a given transition and generalization structure, and finding the best transition and generalization structure consistent with a given alignment of the training data. If we represent the alignment by associating with each snapshot-action pair a probability distribution over the partition sets, we can immediately reduce the iterative algorithm to the Baum-Welch algorithm, an expectation-maximization (E-M) algorithm used to induce discrete Hidden Markov Models from sequences of symbols [1]. We interpret the E-step as a way of determining the best alignment of the sequences relative to a given HMM, and the M-step as a way of inducing the best procedure model given an alignment. For PBD applications, where user actions are often dependent on the current “place” within the procedure and on the content of the screen, it is additionally necessary to induce predictive mappings from inputs (snapshots) to outputs (actions) and next states. Bengio and Frasconi [6] introduced an extension to HMMs, called the Input-Output Hidden Markov Model, or IOHMM, that satisfies this assumption. IOHMMs predict the next state and the next output symbol as a function of the current state and of the current input symbol.

The third source of constraint above cannot be employed by either HMM or IOHMM algorithms, yet as we discussed it is a natural form of knowledge in the PBD domain. These algorithms do not take any form of explicit knowledge about the domain, rather they rely entirely on the dataset provided.

SimAlignGen algorithms extend Hidden Markov Model induction by incorporating this similarity domain knowledge as a bias on the alignments which the algorithm considers in its search for a predictive model of the observed data.

We now formally define the SimIOHMM by describing how it differs from the standard IOHMM as described in [6]. We use standard notational convention.

Bengio and Frasconi's IOHMM

The goal of a IOHMM is to model the *conditional* distribution of an output sequence $\mathbf{Y}$ given an input sequence $\mathbf{U}$ as an HMM, i.e., by postulating the existence of a hidden variable, the *state* X , belonging to a finite set $\mathcal{X}$, such that

$$\mathbb{P}[(\mathbf{U}_t, \mathbf{Y}_t, X_t) \mid \{(\mathbf{U}_j, \mathbf{Y}_j, X_j)\}_{j=1}^{t-1}] = \mathbb{P}[(\mathbf{U}_t, \mathbf{Y}_t, X_t) \mid X_{t-1}].$$

The same structure is inherited by the conditional distribution of the outputs given the inputs (i.e., $\mathbb{P}[(\mathbf{Y}_t, X_t) \mid \mathbf{U}_t, \{(\mathbf{U}_j, \mathbf{Y}_j, X_j)\}_{j=1}^{t-1}]$) that can now be written as $\mathbb{P}[(\mathbf{Y}_t, X_t) \mid \mathbf{U}_t, X_{t-1}]$. Note that this probability can be further decomposed as $\mathbb{P}[X_t \mid \mathbf{U}_t, X_{t-1}] \mathbb{P}[Y_t \mid X_t, \mathbf{U}_t, X_{t-1}] = \mathbb{P}[X_t \mid \mathbf{U}_t, X_{t-1}] \mathbb{P}[Y_t \mid X_t, \mathbf{U}_t]$, namely, as a conditional transition probability to state X_t given the previous state and the input at time t , and a conditional probability of the output at time t given the state and input at time t . Hence, inducing a IOHMM is equivalent to estimating the initial probability distribution over the states, $\mathbb{P}[X_0]$ (note that the first input is observed at $t = 1$), the transition probabilities $\mathbb{P}[X_t \mid \mathbf{U}_t, X_{t-1}]$, and the conditional output probabilities $\mathbb{P}[Y_t \mid X_t, \mathbf{U}_t]$. The transition and output probabilities are assumed to be time-independent, namely, for every s and t ,

$$\begin{aligned} \mathbb{P}[X_t = a \mid \mathbf{U}_t = u, X_{t-1} = b] &= \mathbb{P}[X_s = a \mid \mathbf{U}_s = u, X_{s-1} = b] \text{ and} \\ \mathbb{P}[Y_t = y \mid X_t = x, \mathbf{U}_t = u] &= \mathbb{P}[Y_s = y \mid X_s = x, \mathbf{U}_s = u]. \end{aligned}$$

These assumptions make the IOHMM very flexible and yet computationally manageable. The IOHMM allows arbitrarily long time-dependence between the input-output pairs, and is therefore more powerful than a fixed-order Markov model. At the same time, the IOHMM can be efficiently induced from training data using the Baum-Welch algorithm, consisting of two steps: an Expectation step, where the training sequences are aligned to an existing model, and a Maximization step, where the model is updated given the alignment.

The IOHMM E-step efficiently computes a probability distribution over the state space for each input-output pair by means of two steps: the forward recursion computes, for each time t the joint probability of the state at time t and all outputs up to time t given the inputs up to time t ; the backward recursion computes the probability of the outputs after time t , given the state at time t and the inputs from time t on. The results of the forward and backward recursions are then appropriately combined, for each t , to estimate the probability distribution over the states as well as the posterior transition probabilities.

The M-step efficiently recomputes the initial probability distribution over the states, the conditional transition probabilities given the current input, and the conditional distributions of the output given the current state and input either by maximum likelihood or with a generic likelihood estimation method. Bengio and Frasconi followed the latter approach, and used neural networks in the M-step. The efficiency of the M-step stems from the fact that the *global* maximization of the likelihood is performed by *separately* maximizing the likelihoods at each individual state, namely, by finding the parameters that maximize the transition

and output probabilities for each state given the results of the expectation step.

The SimIOHMM

The SimIOHMM extends the IOHMM by further incorporating the bias described earlier in this Section. To this end, each hidden Markov state is associated with one or more representative inputs, as well as with a transition and output distribution. The representative inputs come into play during the E-step and are updated during the M-step, as follows.

The E-Step. The forward recursion is

$$\alpha_{i,t} = \mathbb{P}[\mathbf{y}_t \mid X_t = i, \mathbf{u}_t] \mathbb{S}(\mathbf{u}_t, \mathbf{v}_i) \sum_{\ell \in \mathcal{X}} \phi_{i,\ell}(\mathbf{u}_t) \alpha_{\ell,t-1}, \quad (1)$$

which adds to [6, Equation(20)] the additional term $\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i)$, the similarity score between the input $\mathbf{u}_t$ and the representative sample $\mathbf{v}_i$ of state i , that provides the required bias. Similarly, the backward recursion is

$$\beta_{i,t} = \sum_{\ell \in \mathcal{X}} \mathbb{S}(\mathbf{u}_{t+1}, \mathbf{v}_\ell) \mathbb{P}[\mathbf{y}_{t+1} \mid X_{t+1} = \ell, \mathbf{u}_{t+1}] \phi_{\ell,i}(\mathbf{u}_{t+1}) \beta_{\ell,t+1}, \quad (2)$$

which again adds a bias term to [6, Equation(22)].¹

The bias term $\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i)$, a similarity score, is designed to concentrate the distribution over the states at time t onto those states having representative samples similar to $\mathbf{u}_t$. For sake of simplicity, assume that each state has a unique representative sample. Then $\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i)$ is computed as follows: first, the distances $\{d(\mathbf{u}_t, \mathbf{v}_i)\}_{i=1}^{|\mathcal{X}|}$ between $\mathbf{u}_t$ and the representatives of the states are computed; then these distances are converted into a similarity score by means of a kernel $K(\cdot)$ (for example, a finite-support decreasing function or a Gaussian):

$$\mathbb{S}(\mathbf{u}_t, \mathbf{v}_i) = K(d(\mathbf{u}_t, \mathbf{v}_i)).$$

The definition of $\mathbb{S}(\cdot, \cdot)$ can be obviously extended to capture the similarities between input-output pairs, rather than between inputs. Adding the bias term substantially improves the training time and can improve the classification accuracy, as illustrated in the experiments section.

The M-step. The M-step of the SimIOHMM consists of the M-Step of the IOHMM, plus the recomputation of representative samples. For each HMM state, the sample with highest alignment probability becomes the new representative.

4 Experimental Results

We evaluate our *SimAlignGen* in two ways. We evaluate the utility of using a similarity bias for learning HMM models in a general setting using synthetic data where we can vary the “correctness” of similarity data presented. We then evaluate the predictive performance of this approach in a practical PBD setting implemented on the Microsoft Windows GUI where we measure the effectiveness of the *SimAlignGen* approach as well as the utility of the similarity bias.

¹ The small discrepancies with the actual Equation [6, Equation(22)] are due to typographical errors in the original paper.

Experiment 1: Effectiveness of similarity knowledge

We measure the accuracy of the SimIOHMM as a function of the correctness of the similarity information presented, and compare it to the accuracy of the IOHMM. We expect the performance of *SimAlignGen* algorithms to vary as a function of how well the similarity knowledge matches the process underlying the generated data. We quantify this degree of match by introducing a *similarity correctness* score. Given dataset generated by an HMM and a similarity function, we compute the similarities between each sample and the other samples in the dataset, and find its nearest neighbor. We then define the similarity correctness as the fraction of samples that were generated by the same HMM state as their nearest neighbor. Thus a score of 100% would imply that is possible to partition the dataset points into sets corresponding to the generating Markov nodes using the similarity function alone. Lesser scores are associated with increasing levels of “noise”, or misinformation regarding the underlying generating process.

The data for the experiments is constructed as follows. We start with an HMM template with 10 hidden states. We instantiate an HMM from the template by randomly generating, for each node, a probability distribution on input (binary) feature vectors, a conditional probability distribution on the (binary) outputs given the inputs, and a conditional transition distribution given the inputs. The additional feature used to compute the similarity score is the index number for the state generating each data point to which a zero-mean Gaussian perturbation of chosen variance is added. Figure 1 shows the accuracy of

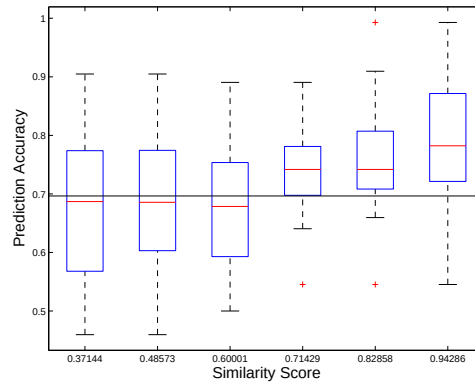


Fig. 1. Accuracy as a function of similarity score

SimIOHMM as a function of the correctness of the provided similarity knowledge. The graph is generated from 180 runs of the SimIOHMM obtained from three-fold cross validation of 60 data sets from different randomly generated HMM models. Each dataset is composed of 400 data-points (20 traces with 20 snapshot-action pairs in each) along with a varying Gaussian noise added to the similarity feature. A similarity correctness score and an accuracy were computed

for each run. The resulting points are binned according to similarity score and then averaged to produce the predictive accuracies reported in the box plot. The box plot shows the median, upper and lower quartile as well as the maximum and minimum accuracies obtained at each level.

The horizontal line just below 70% represents the average comparative IOHMM performance (which does not take into account similarity knowledge). As expected *SimAlignGen* outperforms IOHMMs when the similarity bias provides an accurate model of the underlying HMM. Conversely, learning performance is degraded by a similarity bias that does not represent well the model generating the data. In this experiment the cross-over point between the performance of the two algorithms is between 60% and 70% similarity correctness.

This result raises the question of what kind of correctness scores can we expect in practical applications to programming by demonstration. To investigate this question we used our system to observe eleven Microsoft Window®’s users whose task was to modify the DNS settings of the machine according to written instruction. We then annotated each snapshot-action pair with a label that specifies the corresponding documentation step, and computed the similarity correctness score for the Windows®-specific similarity function implemented in our system (This similarity functions combines several factors, including the previous action taken by the user, and the text on the title bar of the window with focus, etc). We finally obtained a similarity correctness of 88%, much above the 60-70% cross over point for the two algorithms. This is not unsurprising: different parts of an underlying application will intentionally have many redundantly distinguishing GUI features as cues to the user. *SimAlignGen* is a novel approach to leveraging those redundant cues towards inducing a procedure model.

Experiment 2: SimIOHMM training time

In our second set of experiments we measure the performance of the SimIOHMM as part of a PBD system for capturing procedures on the Microsoft Windows GUI. In addition to the gains in accuracy demonstrated above, we show that SimIOHMM can result in substantial reductions in training time and better scalability as a function of training set size when compared with IOHMMs. Figure 2(a) shows the average training time verses number of training traces.

In the figure, it is apparent that the from the viewpoint of training time, the SimIOHMM scales much better than the IOHMM, and that the ratio of the IOHMM training time to the SimIOHMM training time is superlinear in the number of training traces. The faster training time of the SimIOHMM is due to the fact that the training sets used to train the classifiers tend to be smaller. The main reason is that a snapshot-action pair can be aligned only with states having similar representative samples. A way of measuring this effect is by analyzing the dispersion of the alignment distributions $\gamma_n(t)$ for the training traces once convergence is reached. A measure of dispersion of a probability distribution is its entropy. Figure 2 (b) shows the average entropy (in bits) of the alignment at convergence as a function of the number of training traces. The experiments are the same used for Figure 2 (a). Due to the similarity bias, the SimIOHMM yields substantially more concentrated alignment probabilities than the IOHMM, and

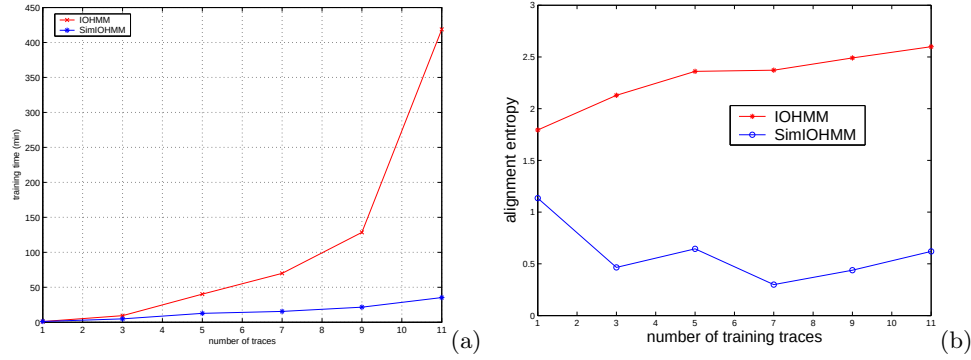


Fig. 2. (a) Training time for IOHMM and SimIOHMM as a function of the number of training traces. (b) Alignment entropy (in bits) for IOHMM and SimIOHMM as a function of the number of training traces.

the difference between these entropies is an increasing function of the number of traces. These findings are also confirmed by the analysis of simulated data.

5 Conclusions

This paper presents an approach to procedure model induction based on the idea of similarity-based alignment and generalization, and makes the following contributions: (i) a novel approach to PBD based on similarity-based alignment and generalization; (ii) the *SimAlignGen* class of algorithms that extend traditional sequence alignment algorithms by the addition of a third bias based on a *similarity metric*; (iii) an instance of an *SimAlignGen* algorithm, called SimIOHMM, which has been implemented as part of a programming by demonstration system on the Windows platform; and (iv) an empirical evaluation showing accuracy improvements as a function of synthetic similarity data, and large efficiency improvements over traces collected from a real-world procedure.

References

1. Rabiner, L.R., Juang, B.H.: An introduction to Hidden Markov Models. IEEE ASSP Magazine (1986) 4–15
2. Cypher, A., ed.: Watch what I do: Programming by demonstration. MIT Press, Cambridge, MA (1993)
3. Lieberman, H., ed.: Your Wish is My Command: Giving Users the Power to Instruct their Software. Morgan Kaufmann (2001)
4. Lau, T., Domingos, P., Weld, D.S.: Version space algebra and its application to programming by demonstration. In: Proc. Seventeenth Int. Conf. on Machine Learning. (2000) 527–534
5. Mulsby, D., Witten, I.H.: Cima: an interactive concept learning system for end-user applications. Applied Artificial Intelligence **11** (1997) 653–671
6. Bengio, Y., Frasconi, P.: Input-Output HMM's for sequence processing. IEEE Trans. Neural Networks **7** (1996) 1231–1249